

Part 4.1

Tools Overview and Project Architecture

This part covers the complete production deployment stack. Each tool has a specific, non-overlapping responsibility:

Tool	Role	Analogy
Terraform	Provisions infrastructure (the VPS exists)	Building the plot of land
Ansible	Configures the server and deploys the app (the server is ready)	Constructing the building on that land
Docker Compose	Runs and manages containers (the app is running)	Furnishing the building
Caddy	Reverse proxy + automatic HTTPS (the app is reachable via domain)	The front door and reception
Uptime Kuma	Monitors if endpoints are up or down	The smoke alarm
Prometheus	Collects and stores numeric metrics over time	The health monitor recording vitals
Grafana	Visualizes Prometheus data as dashboards	The screen displaying those vitals
Grafana k6	Load tests to simulate concurrent user traffic	The stress test
Sublyzer	Application-level observability and error tracking	The doctor analyzing symptoms
Hostinger VPS	The cloud server hosting everything	The actual plot of land

The full pipeline in order:

Terraform → Ansible → Docker Compose → Caddy → (Monitoring Stack)

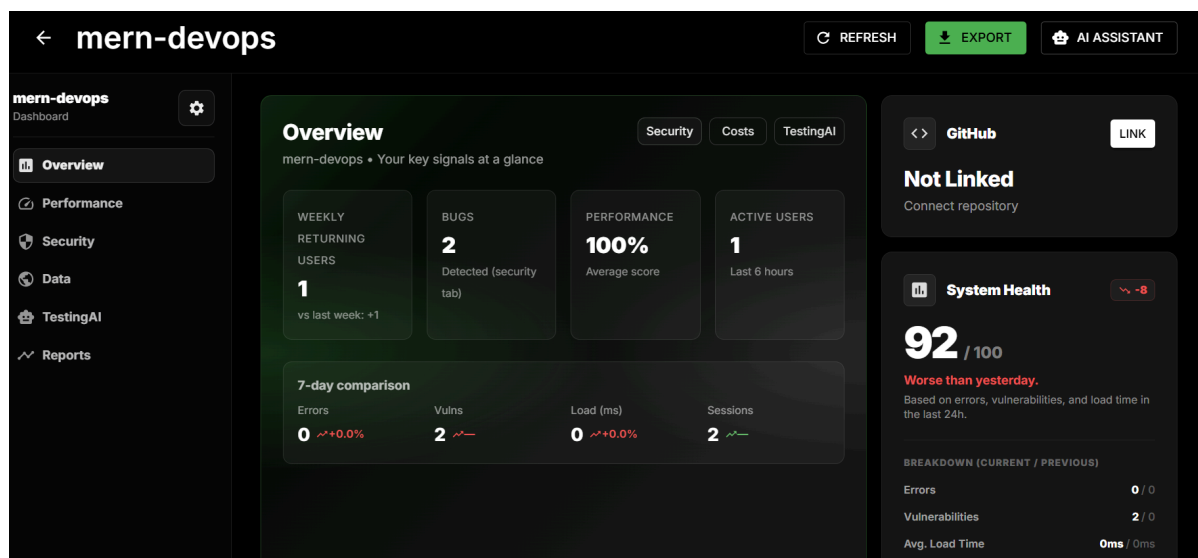
Sublyzer

Sublyzer is an observability and monitoring tool that gives you a real-time dashboard tracking your application's health, performance, and errors directly from the user's perspective.

Setup Steps

1. **Create an account** on the Sublyzer platform and create a new project — this tells the system "I want to monitor this specific application."
2. **Get the integration key** — after setting up your project, Sublyzer gives you a unique integration code (like a secret password that lets your app talk to Sublyzer's servers).
3. **Connect the app** — save the Sublyzer SDK file into the project's public folder, then paste the integration code into it. This step "activates" the monitoring.
4. **View the dashboard** — once the app is running, the Sublyzer dashboard immediately starts showing live data.

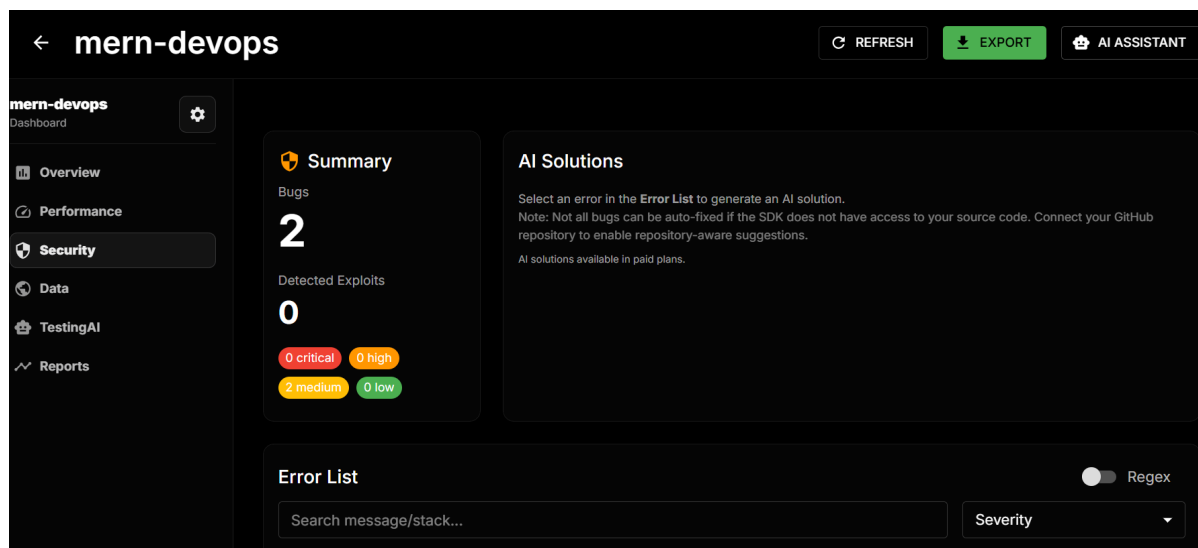
What the Dashboard Shows



The Overview tab shows your key signals at a glance:

- **Weekly Returning Users** — tracks user retention over time
- **Bugs** — number of detected issues (linked to the Security tab)
- **Performance** — average performance score (100% = fast, responsive)
- **Active Users** — who is using the app right now (last 6 hours)
- **System Health** — a composite score out of 100, calculated from errors, vulnerabilities, and load time. A score of 92/100 with "Worse than yesterday" means something degraded slightly — worth investigating.
- **7-day comparison** — shows trends for Errors, Vulnerabilities, Load time, and Sessions

The Security Tab — Error Tracking



The Security tab is where errors are tracked. It shows:

- **Summary** — total bug count and detected exploits broken down by severity (critical / high / medium / low)
- **AI Solutions** — if you connect your GitHub repository, Sublyzer can analyze errors against your actual source code and suggest fixes automatically
- **Error List** — searchable list of every error that occurred, with the full stack trace and severity rating

Key insight: You connect GitHub to Sublyzer to enable repository-aware AI suggestions. Without GitHub access, the AI can still suggest generic fixes but cannot reference your specific code.

Terraform

What is Infrastructure as Code (IaC)?

Terraform is a tool for **Infrastructure as Code** — instead of manually clicking through a cloud console to create servers, DNS records, and networks, you write `.tf` configuration files that describe what infrastructure should exist. Terraform reads those files and makes it happen via cloud provider APIs.

Why IaC matters:

- **Reproducibility** — spin up identical environments in dev, staging, and production from the same code

- **Version control** — infrastructure changes live in Git: reviewable, reversible, auditable
- **Automation** — CI/CD pipelines can provision and tear down infrastructure automatically
- **No snowflake servers** — without IaC, every manually configured server ends up slightly different. With Terraform, they are always identical.

Provisioning vs. Configuration Management (Critical Distinction)

This is one of the most common DevOps interview questions:

Provisioning (Terraform): Creates, changes, or destroys infrastructure. Answers "what should exist?" — servers, networks, DNS records, load balancers.

Terraform talks to **cloud provider APIs** and keeps state in `terraform.tfstate`.

Configuration Management (Ansible): Turns a provisioned server into a ready deployment target. Answers "what should be installed and running on that server?" — Docker, Git, app configs. Ansible connects over **SSH** and executes tasks on the server.

Terraform output (VPS IP address) → becomes Ansible input (inventory target)

The Terraform CLI Workflow

The CLI is the bridge between your local `.tf` files and the cloud provider. It reads your configuration, interacts with provider APIs, and manages the full lifecycle.

```
.tf config files (desired state)
  ↓
terraform init      → download provider plugins, prepare directory
terraform fmt       → auto-format code to standard style
terraform validate  → check syntax and logic (no API calls made)
terraform plan      → calculate diff: what will be created /
changed / destroyed
terraform apply      → execute the plan, build real infrastructure
terraform output     → print outputs (e.g., VPS IP) for downstream
tools
```

`terraform init`

The mandatory first step. Downloads the required provider plugins and creates the `.terraform.lock.hcl` file. You must run this before any other command. Like `npm install` — it sets up the environment.

```
terraform init
# Initializing provider plugins...
# - Finding hostinger/hostinger versions matching "0.1.22"...
# Terraform has been successfully initialized!
```

`terraform fmt`

Automatically formats your `.tf` files to follow standard Terraform style — consistent indentation, spacing, and alignment. Run this before every commit.

```
terraform fmt
```

`terraform validate`

Checks that your configuration files are syntactically correct and logically valid before making any real changes. Does not call any cloud APIs — it only reads your local files.

```
terraform validate
# Success! The configuration is valid.
```

`terraform plan`

The most important safety step. Generates a "what-if" analysis showing exactly what Terraform would create, modify, or destroy — **without actually doing anything**. Always read this carefully before applying.

```
terraform plan
# Resource actions are indicated by symbols:
#   + = will be created
#   ~ = will be modified in-place
#   - = will be destroyed
#   -/+ = will be destroyed and recreated (forces replacement)
```

`terraform apply`

Executes the plan and builds or modifies real infrastructure. Asks for confirmation (yes) unless you pass `-auto-approve` .

```
terraform apply
terraform apply -auto-approve    # Skip the confirmation (used in
CI/CD)
```

terraform output

After infrastructure is provisioned, prints the output values you defined (like the VPS IP address). These values are often passed to downstream tools like Ansible.

```
terraform output
# vps_ip = "<YOUR_SERVER_IP>"
# vps_id = "1618885"
```

Terraform Block Types

Infrastructure in Terraform is defined through four main block types:

Terraform Block → Meta settings. Defines what provider plugins are needed and which versions to lock. This is the project's "rules" file.

```
terraform {
  required_providers {
    hostinger = {
      source = "hostinger/hostinger"
      version = "0.1.22"
    }
  }
}
```

Provider Block → Tells Terraform how to authenticate and communicate with the target cloud platform. Each cloud (AWS, Hostinger, GCP) has its own provider.

```
provider "hostinger" {
  api_token = var.hostinger_api_token    # Never hardcode the actual
token here
}
```

Resource Block → The most important block. Defines the actual infrastructure objects you want Terraform to create and manage — VPS instances, DNS records, virtual networks. This is the core logic.

```
resource "hostinger_vps" "prod" {  
  plan          = var.plan  
  data_center_id = var.data_center_id  
  template_id   = var.template_id  
  hostname      = var.hostname  
  password      = var.root_password  
}
```

Variable Block → Defines inputs that can be customized per environment. Keeps sensitive and environment-specific values out of the main logic files.

```
variable "hostname" {  
  type      = string  
  description = "this is my hostname"  
}
```

Output Block → Prints important values after deployment and makes them available for downstream tools.

```
output "vps_ip" {  
  value = hostinger_vps.prod.ipv4_address  
}
```

Terraform File Structure

```
ops/  
└─ terraform/  
    └─ prod/  
        └─ main.tf          ← Core resource and provider  
definitions  
        └─ variables.tf     ← Variable declarations (the  
"shape")  
        └─ outputs.tf       ← Output definitions (the  
"return values")  
        └─ terraform.tfvars ← Actual variable values –
```

NEVER commit to Git

└─ terraform.tfvars.example ← Documentation template –
safe to commit

└─ .terraform.lock.hcl ← Provider version lock –
commit this

└─ terraform.tfstate ← State file – NEVER commit,
store remotely

main.tf — The Primary Entry Point

Contains the core resource definitions. Anything that describes *what infrastructure to build* lives here. If it grows beyond ~150 lines, split resources into logically named files (`network.tf` , `iam.tf`).

```
# main.tf

terraform {
  required_providers {
    hostinger = {
      source  = "hostinger/hostinger"
      version = "0.1.22"
    }
  }
}

provider "hostinger" {
  api_token = var.hostinger_api_token
}

# Data sources: read-only lookups of existing cloud data
data "hostinger_vps_data_centers" "all" {}
data "hostinger_vps_templates" "all" {}
data "hostinger_vps_plans" "all" {}

# Resource: the actual VPS to provision
resource "hostinger_vps" "prod" {
  plan           = var.plan
  data_center_id = var.data_center_id
  template_id    = var.template_id
  hostname       = var.hostname
}
```



```
password      = var.root_password
}
```

Best practices:

- Keep `main.tf` focused on resource definitions only
- Move variables to `variables.tf` and outputs to `outputs.tf` — don't mix them in `main.tf`
- Avoid "Terraliths" — don't manage completely unrelated systems in one file

variables.tf — Input Declarations

This file **declares** variables — it defines their name, type, and optionally a default value or description. It does NOT assign actual values (that happens in `terraform.tfvars`).

```
# variables.tf

# Required variable – no default means Terraform FORCES you to
# provide it
variable "hostinger_api_token" {
  type        = string
  description = "API token from Hostinger control panel"
  sensitive   = true    # Prevents the value from being printed in
  # CLI output
}

# Optional variable – has a default value as a fallback
variable "hostname" {
  type        = string
  description = "Hostname for the VPS"
  default     = "prod-server-1"
}

variable "plan" {
  type    = string
  default = "hostingerin-vps-kvm1-inr-1m"
}

variable "data_center_id" {
  type = number
```

```

    default = 13      # 13 = Mumbai data center
}

variable "template_id" {
    type      = number
    default   = 1200
}

variable "root_password" {
    type        = string
    sensitive   = true      # Hides from CLI output
}

```

Supported variable types:

- `string` — Text: `"us-east-1"`
- `number` — Numeric: `3`, `5.5`
- `bool` — True/false: `true`, `false`
- `list` — Ordered collection: `["a", "b", "c"]`
- `map` — Key-value pairs: `{ env = "prod", region = "us" }`

Required vs. Optional:

- Declare without `default` → **required** (Terraform errors if not provided)
- Add `default = "value"` → **optional** (falls back to the default if not provided)
- Use `sensitive = true` for passwords and tokens → value is hidden from all CLI output

Interview question: "How do you write a required input in Terraform?"

Answer: By declaring a variable without a `default` value. Terraform will force the user to provide that input at runtime.

Interview question: "How do you make a variable optional?" Answer: By adding a `default` value to its declaration. The default acts as a fallback if no value is provided.

`outputs.tf` — The Return Values

Outputs serve three purposes:

1. **CLI Feedback** — Display important information (like VPS IP) in your terminal after `terraform apply`
2. **Downstream Tools** — The VPS IP output becomes the target in Ansible's inventory
3. **Inter-Module Communication** — Pass data between Terraform modules in complex setups

```
# outputs.tf

output "vps_ip" {
  description = "The public IPv4 address of the production VPS"
  value       = hostinger_vps.prod.ipv4_address
}

output "vps_id" {
  description = "The Hostinger VPS resource ID"
  value       = hostinger_vps.prod.vps_id
}
```

terraform.tfvars — Actual Values (Never Commit)

Assigns real values to the variables declared in `variables.tf`. Terraform automatically loads this file without you specifying it on the command line.

```
# terraform.tfvars - ADD TO .gitignore

plan           = "hostingerin-vps-kvm1-inr-1m"
data_center_id = 13
template_id    = 1200
hostname       = "mern-prod-server"
root_password  = "your_secure_password_here"
```

Why not hardcode values directly in `main.tf` ?

1. **Environment switching** — have `dev.tfvars`, `staging.tfvars`, `prod.tfvars` and switch with: `terraform apply -var-file="prod.tfvars"`
 2. **Security** — add `*.tfvars` to `.gitignore` so sensitive values never reach GitHub
-

terraform.tfvars.example — Documentation Template (Safe to Commit)

A documentation-only file. Terraform does NOT read it — it is purely for humans reading the repository. It shows other developers exactly which values they need to provide.

```
# terraform.tfvars.example
# Copy this file to terraform.tfvars and fill in your actual
# values.
# DO NOT commit terraform.tfvars - it contains sensitive data.

plan                = "your_plan_slug_here"
data_center_id      = 0           # Run: terraform console >
data.hostinger_vps_data_centers.all
template_id         = 0           # Run: terraform console >
data.hostinger_vps_templates.all
hostname            = "your-server-hostname"
root_password       = "your_secure_root_password"
```

.terraform.lock.hcl — The Dependency Lock File

Ensures every team member uses the **exact same version** of provider plugins. Contains the version number and cryptographic checksums (hashes) to verify the provider code hasn't been tampered with and is identical across operating systems.

Analogy: `package-lock.json` in Node.js — it locks the exact dependency tree.

Rule: Commit this file to Git. It prevents surprise breakage from provider version changes.

terraform.tfstate — The Brain

The most critical file in a Terraform project. It is a JSON file that acts as the **single source of truth**, mapping your HCL resource names to real-world cloud resource IDs.

What it stores:

- **Mapping** — your code says `hostinger_vps.prod` but Hostinger's API sees `1618885`. The state file remembers this link.
- **Metadata** — resource dependencies, generated IPs, resource IDs, and attributes not written in your code but needed for updates
- **Performance** — Terraform reads state before calling cloud APIs, so it only queries what actually needs updating

The state lifecycle:

```
Before first apply → state file is empty
After terraform apply → state file records all created resources
and their IDs
On next terraform plan → Terraform compares: Code vs State File vs
Reality
terraform destroy → deletes resources, clears them from state
```

Critical warnings:

- **Never edit this file manually.** Use `terraform state` commands if you must fix state.
- **The file contains sensitive data in plain text** — resource IDs, sometimes passwords or generated keys.
- **Never commit to a public Git repo.**

Interview question: "Why is storing `terraform.tfstate` on a local laptop bad for a team?" Two reasons:

1. **Split brain** — if two engineers run `terraform apply` simultaneously with separate local state files, their states diverge. Each thinks it owns different things, leading to destructive conflicts.
2. **Security** — laptops are not encrypted to the standard required for a file containing cloud credentials and resource IDs.

Solution: Remote Backends

- **AWS** — store state in an S3 bucket + DynamoDB table for state locking (prevents simultaneous applies)
 - **Terraform Cloud / HCP Terraform** — managed remote state with built-in locking and access controls
-

Handling Sensitive Variables — Environment Variables

Instead of storing the API token in `terraform.tfvars`, set it as an OS environment variable. Terraform automatically reads variables prefixed with `TF_VAR_`.

```
# Windows PowerShell
$env:TF_VAR_hostinger_api_token="your_token_here"

# macOS / Linux
export TF_VAR_hostinger_api_token="your_token_here"
```

Setting it as an environment variable means you don't have to paste it into the terminal on every `terraform plan` run, and it is never written to disk.

Practical Workflow — Provisioning the Hostinger VPS

These are the exact commands run in the course:

```
# Current files: main.tf, variables.tf, outputs.tf

# Clean up stale state if switching provider versions
Remove-Item -Recurse -Force .terraform -ErrorAction SilentlyContinue
Remove-Item -Force .terraform.lock.hcl -ErrorAction SilentlyContinue

# Initialize - downloads hostinger/hostinger v0.1.22 provider
# Note: v0.1.3 had issues; v0.1.22 is the fix
terraform init -upgrade
# Output: Terraform has been successfully initialized!

terraform fmt
terraform validate
# Output: Success! The configuration is valid.

# Set API token via environment variable
$env:TF_VAR_hostinger_api_token="your_token"

# terraform plan - paste hostinger key or use env variable
terraform plan
```

```
# Apply
terraform apply

# Get outputs
terraform output
```

Exploring available data center IDs

```
terraform console
> data.hostinger_vps_data_centers.all.data_centers
# Returns:
# {
#   "city" = "Mumbai"
#   "continent" = "Asia"
#   "id" = 13
#   "location" = "in"
#   "name" = "mum"
# }
```

Importing an Existing VPS (instead of creating a new one)

`terraform import` tells Terraform: "This resource already exists in the cloud — start managing it in state."

```
# Format: terraform import <resource_address> <cloud_resource_id>
terraform import hostinger_vps.prod 1618885

# Output:
# hostinger_vps.prod: Importing from ID "1618885"...
# hostinger_vps.prod: Import prepared!
# hostinger_vps.prod: Refreshing state... [id=1618885]
# Import successful!
# The resources that were imported are shown above.
# These resources are now in your Terraform state.
```

After importing, running `terraform plan` shows the diff between your `.tf` file and the real resource's current configuration. In the course, this showed:

```
-/+ resource "hostinger_vps" "prod" {
  ~ data_center_id = 23 -> 13      # forces replacement
  ~ template_id    = 1077 -> 1200 # forces replacement
```

```
+ password = (sensitive value)    # forces replacement
~ plan = "KVM 1" -> "hostingerin-vps-kvm1-inr-1m"
}
Plan: 1 to add, 0 to change, 1 to destroy.
```

The `-/+` symbol means the resource must be **destroyed and recreated** because the change to `data_center_id` cannot be done in-place.

`terraform.tfstate` is created automatically after the first `terraform apply` or `terraform import`. You will see the file appear in your project directory.

Ansible

What is Ansible?

Ansible is an open-source IT automation engine used to configure servers and deploy applications. Where Terraform provisions the infrastructure (server exists), Ansible makes it ready (server is configured and app is deployed).

Formal definition: An automation tool for remote machines that is reachable, repeatable, and targeted. It provides consistency, confidence in deployment processes, and auditability.

Why Ansible Instead of Manual SSH Commands?

Before understanding Ansible, you need to understand what problem it solves over plain SSH.

SSH (Secure Shell) lets you connect to a remote machine and run commands. Every `ssh root@<YOUR_SERVER_IP>` gives you a remote shell. It works, but:

- **Not repeatable** — SSH commands are one-time, manual, and undocumented
- **Not scalable** — running the same setup commands on 50 servers manually is impractical
- **Not idempotent** — running `apt install docker.io` twice on the same server either errors or reinstalls unnecessarily
- **No audit trail** — there is no record of what was done to which server and when

Ansible replaces ad-hoc SSH commands with **YAML playbooks** — structured, repeatable, version-controlled automation that works the same way every time.

Core Concepts

Agentless Architecture Unlike tools like Chef or Puppet, Ansible requires **no software installed on target machines**. It connects via standard SSH (Linux) or WinRM (Windows), runs its tasks, and then cleans up. This makes setup dramatically simpler.

Control Node vs. Managed Nodes

- **Control Node** — the machine where Ansible is installed and from which you run commands. Your laptop (WSL) or a CI/CD runner.
- **Managed Nodes** — the remote servers being automated. In this course, the Hostinger VPS provisioned by Terraform.

Inventory A file listing all the managed nodes Ansible knows about, organized into groups. Instead of hardcoding IPs into every script, you define them centrally.

```
# inventory/hosts
[prod]
<YOUR_SERVER_IP> ansible_user=root
ansible_ssh_private_key_file=~/.ssh/id_ed25519

[stage]
192.168.1.100 ansible_user=ubuntu
```

Groups (like `prod` , `stage` , `web` , `db`) let you target specific environments precisely — run a deployment only against `prod` without touching `stage` .

Playbooks Human-readable YAML files that define the desired state of a system. A sequence of tasks to execute on target nodes. They serve as both automation and documentation.

```
---
- name: Bootstrap production server
  hosts: prod           # Target "prod" group from inventory
  become: true         # Run tasks as root

  tasks:
    - name: Update package cache
      apt:
        update_cache: yes
```

```

- name: Install Docker
  apt:
    name: docker.io
    state: present    # "present" = install if not already there

- name: Start Docker service
  service:
    name: docker
    state: started
    enabled: yes

```

Modules The tools in Ansible's toolbox — small programs that perform specific tasks on managed nodes. Each module is designed to be idempotent: it only makes a change if the system is not already in the desired state.

Module	What It Does
apt / yum	Install or remove packages
copy	Copy files from control node to managed node
template	Copy a Jinja2 template file with variable substitution
service	Start, stop, restart, or enable system services
shell / command	Run arbitrary shell commands
get_url	Download files from a URL
file	Create directories, set permissions, manage symlinks

Idempotency Running the same Ansible playbook 10 times produces the same result as running it once. If Docker is already installed, the `apt` module returns `ok` and skips installation. This is what makes Ansible safe to run repeatedly — in CI/CD pipelines, on every deploy, or whenever you want to ensure a server's state is correct.

become: true — Privilege Escalation

```

become: true          # Tells Ansible to escalate privileges
become_method: sudo   # How to escalate (sudo is the default)
become_user: root      # The user to become (root is the default)

```

Many tasks (installing packages, modifying system files, creating users) require root permissions. `become: true` handles this automatically — it is the equivalent

of prefixing every shell command with `sudo` .

Ansible Output: Task Status

After every task, Ansible reports one of three statuses:

Status	Color	Action Taken?	Meaning
ok	Green	No	Task succeeded; system was already in the desired state
changed	Yellow	Yes	Task succeeded and a modification was made
failed	Red	No/Partial	An error occurred; Ansible stops further tasks on this host

Why `ok` is the goal: Seeing all `ok` on a re-run means every server is already in the desired state — idempotency working perfectly.

Practical tips:

- `ignore_errors: true` — lets the playbook continue even if a specific task fails
- Handlers only fire on `changed` — a "restart nginx" handler won't trigger if the config file didn't change

Project Directory Structure

```
ops/
├── ansible/
│   ├── ansible.cfg                # Project-specific config - avoids
passing flags every time
│   ├── inventory/
│   │   ├── hosts                 # List of servers and groups
│   │   ├── group_vars/
│   │   │   ├── all.yml           # Variables applied to ALL groups
│   │   └── host_vars/
│   │       └── <YOUR_SERVER_IP>.yaml # Variables for a specific
```

```

host
├── playbooks/
│   ├── bootstrap.yml      # Makes raw server ready
│   ├── deploy.yml         # Deploys the application
│   └── observability.yml   # Deploys the monitoring stack
├── roles/                  # Reusable, modular logic
│   ├── common/
│   │   ├── tasks/         # Main task files
│   │   └── handlers/      # Service restarts (trigger on
│   │                       "changed")
│   ├── templates/         # Jinja2 templates (.j2)
│   ├── files/             # Static files to copy
│   └── vars/               # Role-specific variables
└── webserver/

```

Why this structure?

- `ansible.cfg` — stores the inventory path, SSH key, and remote user so you don't pass `--inventory` and `--user` flags on every command
- Separating `group_vars/` and `host_vars/` from `playbooks/` — configuration data is decoupled from execution logic, keeping playbooks clean and DRY (Don't Repeat Yourself)
- `roles/` — reusable modules of automation that can be shared across different playbooks

Setting Up SSH Keys and Running Playbooks

```

# Install Ansible on WSL
sudo apt update
sudo apt install -y ansible

# Navigate to project root (Windows drives are accessible under
# /mnt/ in WSL)
cd /mnt/d/Coding/devops/"Final Part - 2026"

# Generate a dedicated SSH key for Ansible
# -t ed25519: modern elliptic curve algorithm (faster and more
# secure than RSA)
# -a 64: 64 rounds of key derivation (makes brute-force much
# harder)

```

```
# -C: a label/comment to identify the key
ssh-keygen -t ed25519 -a 64 -f ~/.ssh/id_ed25519 -C "devops-course-ansible"

# Copy the public key to the VPS to authorize passwordless login
ssh-copy-id -i ~/.ssh/id_ed25519.pub root@<YOUR_SERVER_IP>

# Verify SSH connection manually
ssh -i ~/.ssh/id_ed25519 root@<YOUR_SERVER_IP>

# Point Ansible to your project's config file
export ANSIBLE_CONFIG="$PWD/ansible.cfg"

# Verify Ansible version and config
ansible --version | sed -n '1,6p'

# Test connectivity (ping all hosts in "prod" group)
ansible prod -m ping

# Run the bootstrap playbook (install Docker, Git, etc.)
ansible-playbook bootstrap.yml

# Run the deploy playbook
ansible-playbook deploy.yml

# Run the observability playbook (deploy Prometheus, Grafana, etc.)
ansible-playbook observability.yml
```

Note on WSL paths: In WSL, your Windows D:\ drive is accessible as /mnt/d/. So D:\Coding\devops\... becomes /mnt/d/Coding/devops/...

The Ansible Deployment Flow

```
Your Laptop (Control Node – runs Ansible)
  ↓ SSH
Inventory → select "prod" group (<YOUR_SERVER_IP>)
  ↓
1. bootstrap.yml      → install Docker, Docker Compose, Git
                       → create app directory (/opt/statusboard)
  ↓
2. deploy.yml         → git clone / git pull latest code
```

```

→ copy .env files
→ docker compose up -d --build
↓
3. verify → curl http://VPS_IP/api/health
↓
VPS (Managed Node) – app is running

```

Caddy

What is Caddy?

Caddy is a modern web server and **reverse proxy** that sits at the edge of your infrastructure — it is the first thing public internet traffic hits. Its defining feature is **automatic HTTPS**: you give it a domain name and it requests, installs, and renews SSL certificates from Let's Encrypt entirely on its own, with no manual steps.

Why Caddy instead of Nginx for this setup?

With Nginx, setting up HTTPS requires:

1. Installing and configuring Certbot separately
2. Running a cron job to auto-renew certificates every 90 days
3. Debugging renewal failures and certificate errors manually

Caddy eliminates all of this. HTTPS "just works."

Caddy's Role in the Stack

```
graph TD; UserBrowser[User Browser] --> DuckDNS[DuckDNS (DNS) → resolves devopscourseankit.duckdns.org → VPS IP (<YOUR_SERVER_IP>)]; DuckDNS --> VPS[VPS port 80 / 443]; VPS --> Caddy[Caddy Container (reverse proxy)]; Caddy --> Containers["| /          → web container   (port 5173) |  
| /api*      → server container (port 5000) |"]; style Containers stroke-width:2px
```

```
| status subdomain → uptime-kuma (port 3001) |
```

What DuckDNS is: A free dynamic DNS service that gives you a subdomain like `yourname.duckdns.org` and lets you point it to any IP address. Used here to get a real domain for HTTPS without buying one.

Two domains were set up:

- `devopscourseankit.duckdns.org` → main app
- `statusdevopscourseankit.duckdns.org` → Uptime Kuma status page

Both point to the same VPS IP via DuckDNS. The IP is set in the DuckDNS dashboard.

How Caddy Gets SSL Certificates (Automatic ACME)

1. Caddy starts and reads the Caddyfile — sees `devopscourseankit.duckdns.org`
2. Checks its cert store — no certificate exists yet
3. Contacts Let's Encrypt: "I need a certificate for this domain"
4. Let's Encrypt sends a challenge: "Prove you control that domain"
5. Caddy hosts a temporary verification file on port 80
6. Let's Encrypt fetches it — domain ownership confirmed
7. Let's Encrypt issues the certificate — Caddy installs it automatically
8. All traffic served over HTTPS immediately
9. ~30 days before expiry, Caddy silently renews in the background

The Caddyfile

The `Caddyfile` (created inside the `app` directory) is Caddy's configuration format — far simpler than Nginx:

```
devopscourseankit.duckdns.org {  
  
    handle /api* {  
        reverse_proxy server:5000  
    }  
  
    handle {  
        reverse_proxy web:5173  
    }  
}
```

```

    }
}

statusdevopscourseankit.duckdns.org {
    reverse_proxy uptime-kuma:3001
}

```

Container names (`server` , `web` , `uptime-kuma`) work as hostnames because all services are on the same Docker network.

Docker Compose Integration

```

services:
  caddy:
    image: caddy:latest
    container_name: caddy
    restart: unless-stopped
    ports:
      - "80:80"      # HTTP (also used for Let's Encrypt ACME
challenge)
      - "443:443"    # HTTPS
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro      # Mount config
(read-only)
      - caddy_data:/data                          # Persist TLS
certificates
      - caddy_config:/config                      # Persist config
cache
    networks:
      - app-net

volumes:
  caddy_data:      # CRITICAL: stores SSL certs - must persist across
restarts
  caddy_config:

```

Why persist `caddy_data` ? This volume stores the SSL certificates Caddy obtains from Let's Encrypt. If you lose this volume, Caddy re-requests certificates on the next start. Let's Encrypt has rate limits (5 certificates per domain per week), so losing certs frequently can lock you out.

Deployment via Ansible

Run the deploy playbook after updating the Caddyfile and `compose.yaml` :

```
export ANSIBLE_CONFIG="$PWD/ansible.cfg"
ansible-playbook deploy.yml
```

Troubleshooting Caddy TLS

If the HTTPS domain is not working after deployment:

```
# SSH into the VPS directly
ssh root@<VPS_IP>
cd /opt/statusboard

# Restart Caddy so it retries TLS cert provisioning
docker compose restart caddy

# Watch logs to confirm cert is being issued
docker compose logs -f caddy

# If Caddy is stuck (failed ACME attempts are cached):
docker compose down
docker volume rm statusboard_caddy_data    # Clears old/failed cert
cache
docker compose up -d
# Caddy requests fresh certificates on startup
```

Remotely via Ansible (without SSH-ing in manually):

```
ansible prod -b -m shell -a "cd /opt/statusboard && docker compose
-f compose.yaml down && docker compose -f compose.yaml up -d --
build"
```

Monitoring and Uptime Kuma

Monitoring Architecture Overview

Production monitoring has three distinct concerns — each tool addresses a different question:

Tool	Answers
Uptime Kuma	"Is the service reachable right now? Up or down?"
Prometheus + Exporters	"What are the system's numeric metrics? CPU, RAM, container stats?"
Grafana	"What do those metrics look like visually over time?"
Sublyzer	"What errors are real users experiencing inside the app?"

The **Three Pillars of Observability** in DevOps:

- **Metrics** — numeric values over time (CPU%, disk usage, request count) → Prometheus
- **Logs** — detailed text records of events → Loki, ELK Stack
- **Traces** — the path of a single request through distributed services → Jaeger ("yagger"), Grafana Tempo

This course focuses on the Metrics pillar.

Uptime Kuma

Uptime Kuma is a self-hosted monitoring tool that repeatedly sends HTTP/HTTPS requests to your endpoints at regular intervals to check if they are up or down. It is the "first line of defense" — it tells you immediately when something becomes unreachable.

The key distinction from Prometheus: Uptime Kuma is **synthetic monitoring** — it only knows if an endpoint returns a successful response from the outside. It cannot tell you why it failed (was it CPU? RAM? A crashed container?). For the "why," you need Prometheus.

What Uptime Kuma tracks:

- Up/Down status — real-time endpoint availability
- Response time graphs — latency trends over time
- Incident history — timestamped record of every outage
- Certificate expiry — shows how many days until your SSL cert expires
- Alerting — push notifications via Slack, Email, Discord, Telegram

Uptime Kuma in Docker Compose

```
services:
  uptime-kuma:
    image: louislam/uptime-kuma:latest
    container_name: uptime-kuma
    restart: unless-stopped
    volumes:
      - uptime-kuma-data:/app/data      # Persist monitor configs and
history
    networks:
      - app-net

volumes:
  uptime-kuma-data:
```

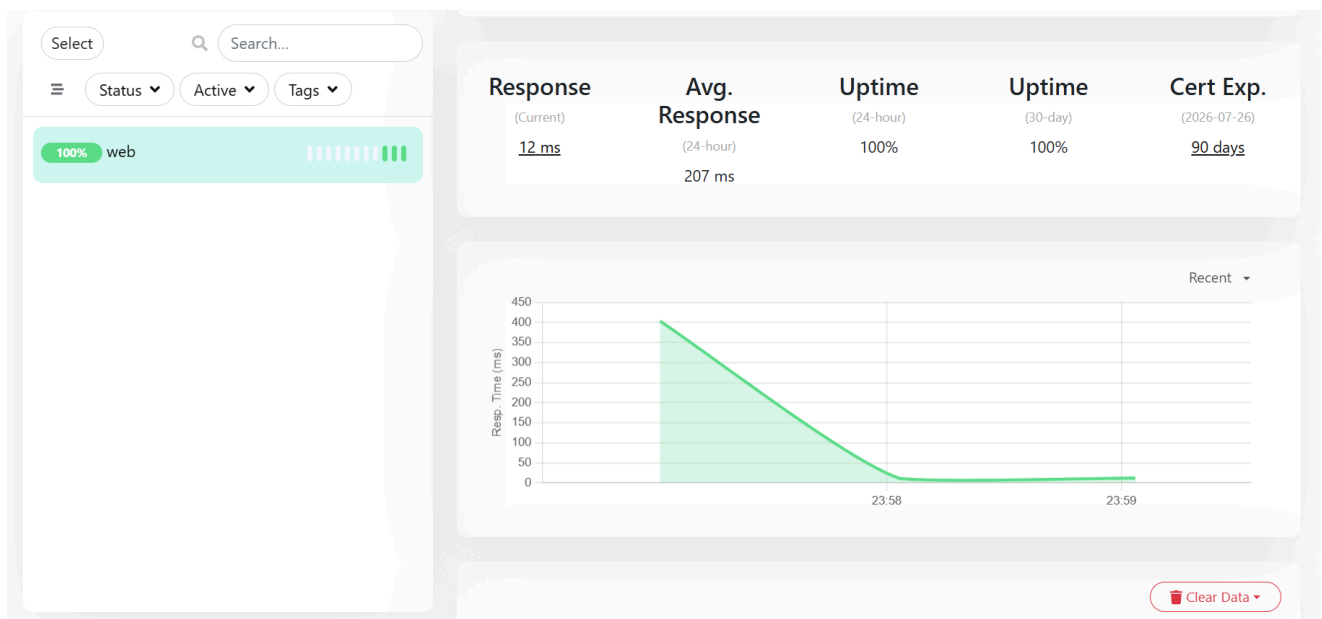
The status subdomain routes through Caddy:

```
statusdevopscourseankit.duckdns.org → Caddy → uptime-kuma:3001
```

Using Uptime Kuma

1. Navigate to `https://statusdevopscourseankit.duckdns.org`
2. Create an admin account with a strong password
3. Add monitors: **New Monitor** → HTTP(s) → enter endpoint URL → set check interval
4. Add monitors for:
 - `https://devopscourseankit.duckdns.org` (frontend)
 - `https://devopscourseankit.duckdns.org/api/health` (backend API)

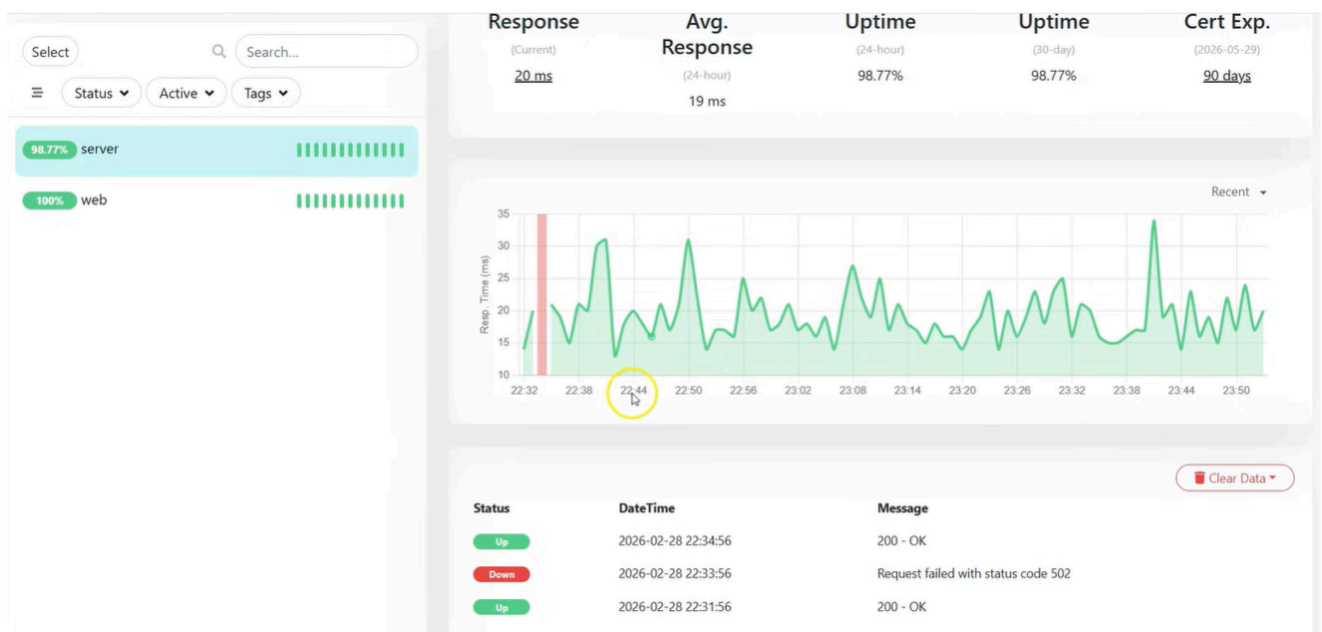
Uptime Kuma Dashboard — Healthy State



The dashboard above shows the `web` monitor at 100% uptime:

- **Response (Current):** 12ms — extremely fast
- **Avg. Response (24-hour):** 207ms
- **Uptime (24-hour):** 100%
- **Uptime (30-day):** 100%
- **Cert Exp.:** 90 days — the SSL certificate is valid for another 90 days
- The response time graph shows a spike when the service first started (the container was initializing), then the latency dropped and stabilized

Uptime Kuma Dashboard — With a Failure



This screenshot shows what a real incident looks like:

- **server** monitor: 98.77% uptime (not 100% — there was a failure)
- **web** monitor: 100% uptime (frontend stayed up)

- The response time graph shows a red bar at 22:33 — the exact moment of failure
- The incident log at the bottom shows:
 - 2026-02-28 22:34:56 — **Up** (200 OK)
 - 2026-02-28 22:33:56 — **Down** (Request failed with status code 502)
 - 2026-02-28 22:31:56 — **Up** (200 OK)

A 502 Bad Gateway from the server endpoint means Caddy received the request but the backend container was not responding — typically a container crash or restart.

Homework: Stop and restart a container while watching Uptime Kuma. Observe how it detects the outage, marks the exact timestamp as "Down," then recovers automatically when the container restarts.

Prometheus and Grafana

Why Prometheus and Grafana Alongside Uptime Kuma?

Uptime Kuma is a **synthetic monitoring tool** — it only checks if an endpoint returns a successful response from the outside. It has no access to internal system data.

Prometheus and Grafana are needed for:

- **Numeric Observability** — CPU saturation, RAM growth, memory usage, container-level CPU throttling
- **Latency Tracking** — identifying performance bottlenecks and slow endpoints
- **Trend Analysis** — historical data to identify patterns like peak traffic times or resource spikes

The difference:

- **Prometheus** — the metrics **database and collector**. Stores time-series data where every metric is a numeric value with a timestamp and labels. Operates on a **pull model**: it actively reaches out to targets every 15 seconds to scrape metrics.
- **Grafana** — the **visualization layer**. Does NOT store any data. Connects to Prometheus, executes PromQL queries, and renders results as graphs,

charts, and dashboards.

Grafana does not store metrics.

It is a display – it reads from Prometheus every time you load a dashboard.

The Exporter / Adapter Concept

Prometheus relies on **exporters** to collect metrics from different systems. An exporter runs alongside the thing being monitored and exposes metrics in a format Prometheus can scrape.

Exporter	Monitors	Port	Category
Node Exporter	Host: CPU, RAM, disk I/O, network	9100	White Box (internal)
cAdvisor	Per-container: CPU, memory, network per container	8080	White Box (internal)
Blackbox Exporter	External HTTP/HTTPS probes, DNS, TCP	9115	Black Box (external)

Node Exporter vs. cAdvisor — why you need both:

- Node Exporter tells you "the host CPU is at 90%" — the big picture
- cAdvisor tells you "the backend container is consuming 85% of CPU" — the drill-down
- You need both: Node Exporter shows there is a problem, cAdvisor shows which container is causing it

White Box vs. Black Box Monitoring

White Box Monitoring — observing internal metrics from inside the system. Uses Node Exporter (host metrics) and cAdvisor (container metrics).

- Catches: CPU spikes, memory leaks, container crashes, disk filling up
- Misses: DNS failures, network routing problems, reverse proxy issues

Black Box Monitoring — probing the system from the outside, exactly as a user would. The Blackbox Exporter sends HTTP/HTTPS requests to your public URLs and reports success, failure, and response time.

- Catches: DNS failures, routing problems, certificate errors, issues that only appear externally
- Misses: internal resource bottlenecks

You need both. Black box tells you the user is experiencing a problem. White box tells you what internally is causing it.

Prometheus Configuration (`prometheus.yml`)

```
global:
  scrape_interval: 15s      # Poll targets every 15 seconds
  evaluation_interval: 15s  # Evaluate alerting rules every 15
seconds

scrape_configs:
  # Prometheus self-monitoring (confirms it is alive and scraping
  correctly)
  - job_name: "prometheus"
    static_configs:
      - targets: ["prometheus:9090"]
        labels:
          instance: "prometheus"

  # Node Exporter – host hardware and OS metrics
  - job_name: "node_exporter"
    static_configs:
      - targets: ["node-exporter:9100"]
        labels:
          instance: "node_exporter"

  # cAdvisor – per-container resource metrics
  - job_name: "cadvisor"
    static_configs:
      - targets: ["cadvisor:8080"]
        labels:
          instance: "cadvisor"

  # Blackbox Exporter – external HTTP probes of public URLs
  - job_name: "blackbox_http"
    metrics_path: /probe
    params:
      module: [http_2xx]
    static_configs:
```

```

- targets:
  - "https://devopscourseankit.duckdns.org"
  - "https://devopscourseankit.duckdns.org/api/health"
  labels:
    job: "blackbox_http"
relabel_configs:
- source_labels: [__address__]
  target_label: __param_target
- source_labels: [__param_target]
  target_label: instance
- target_label: __address__
  replacement: blackbox:9115

```

Why pass the URL for Blackbox specifically? Node Exporter and cAdvisor expose a `/metrics` endpoint at their own port — Prometheus just scrapes those ports directly. The Blackbox Exporter is different: it is an external probe that needs to be told *which URLs to probe*. By passing the URLs in `prometheus.yml`, you tell Prometheus: "Use the Blackbox Exporter to make an HTTP request against this URL and report whether it succeeded."

Blackbox Exporter Configuration (`blackbox.yml`)

```

modules:
  http_2xx:
    prober: http
    timeout: 5s
    http:
      valid_http_versions: ["HTTP/1.1", "HTTP/2.0"]
      valid_status_codes: [200, 201, 204]    # What counts as
"success"
      method: GET
      follow_redirects: true
      tls_config:
        insecure_skip_verify: false        # Validate SSL
properly

```

Time-Series Data Format

Every metric Prometheus stores has three parts:

Metric Name	Labels	Value
Timestamp		

```
node_cpu_seconds_total    {mode="idle", instance="node_exporter"}
1234567.89      1716000000
probe_success            {instance="https://devopscourseankit...",
job="blackbox_http"}    1      1716000015
container_memory_usage_bytes {name="backend", job="cadvisor"}
134217728      1716000015
```

Labels are what make Prometheus powerful — they let you query all containers at once (`container_memory_usage_bytes`) or drill down to a specific one (`container_memory_usage_bytes{name="backend"}`).

Docker Compose for the Observability Stack

(`observability/compose.yaml`)

```
services:
  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    restart: unless-stopped
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - prometheus_data:/prometheus      # Persist time-series
database
  command:
    - '--config.file=/etc/prometheus/prometheus.yml'
    - '--storage.tsdb.retention.time=30d'      # Keep 30 days of
metrics
  networks:
    - monitoring

grafana:
  image: grafana/grafana:latest
  container_name: grafana
  restart: unless-stopped
  ports:
    - "3000:3000"
  volumes:
    - grafana_data:/var/lib/grafana      # Persist dashboards and
settings
```

environment:

- GF_SECURITY_ADMIN_USER=admin
- GF_SECURITY_ADMIN_PASSWORD=your_secure_password

networks:

- monitoring

node-exporter:

image: prom/node-exporter:latest

container_name: node-exporter

restart: unless-stopped

ports:

- "9100:9100"

volumes:

- /proc:/host/proc:ro
- /sys:/host/sys:ro
- /:/rootfs:ro

command:

- '--path.procfs=/host/proc'
- '--path.sysfs=/host/sys'

networks:

- monitoring

cadvisor:

image: gcr.io/cadvisor/cadvisor:latest

container_name: cadvisor

restart: unless-stopped

ports:

- "8080:8080"

volumes:

- /:/rootfs:ro
- /var/run:/var/run:ro
- /sys:/sys:ro
- /var/lib/docker:/var/lib/docker:ro

networks:

- monitoring

blackbox:

image: prom/blackbox-exporter:latest

container_name: blackbox

restart: unless-stopped

ports:

- "9115:9115"

volumes:

- ./blackbox.yml:/config/blackbox.yml:ro

```

command:
  - '--config.file=/config/blackbox.yml'
networks:
  - monitoring

volumes:
  prometheus_data:
  grafana_data:

networks:
  monitoring:
    driver: bridge

```

Why volumes for both Prometheus and Grafana?

- Without a Prometheus data volume, every container restart wipes all collected metrics — all your historical trends are gone
- Without a Grafana data volume, all your custom dashboards and configured data sources are lost on restart

Why a shared network? Within the `monitoring` Docker network, containers reference each other by service name (e.g., `prometheus:9090`, `cadvisor:8080`). Grafana uses `prometheus:9090` as the data source URL — this only resolves because both are on the same network.

Deploying via Ansible (`observability.yml`)

```

---
- name: Deploy observability stack
  hosts: prod
  become: true

  tasks:
    - name: Create observability directory on VPS
      file:
        path: /opt/observability
        state: directory
        mode: '0755'

    - name: Copy docker-compose file
      copy:
        src: ../observability/compose.yaml
        dest: /opt/observability/compose.yaml

```

- **name:** Copy prometheus config
copy:
 - src:** ../observability/prometheus.yml
 - dest:** /opt/observability/prometheus.yml
- **name:** Copy blackbox config
copy:
 - src:** ../observability/blackbox.yml
 - dest:** /opt/observability/blackbox.yml
- **name:** Start observability stack
shell: |
 - cd /opt/observability
 - docker compose up -d

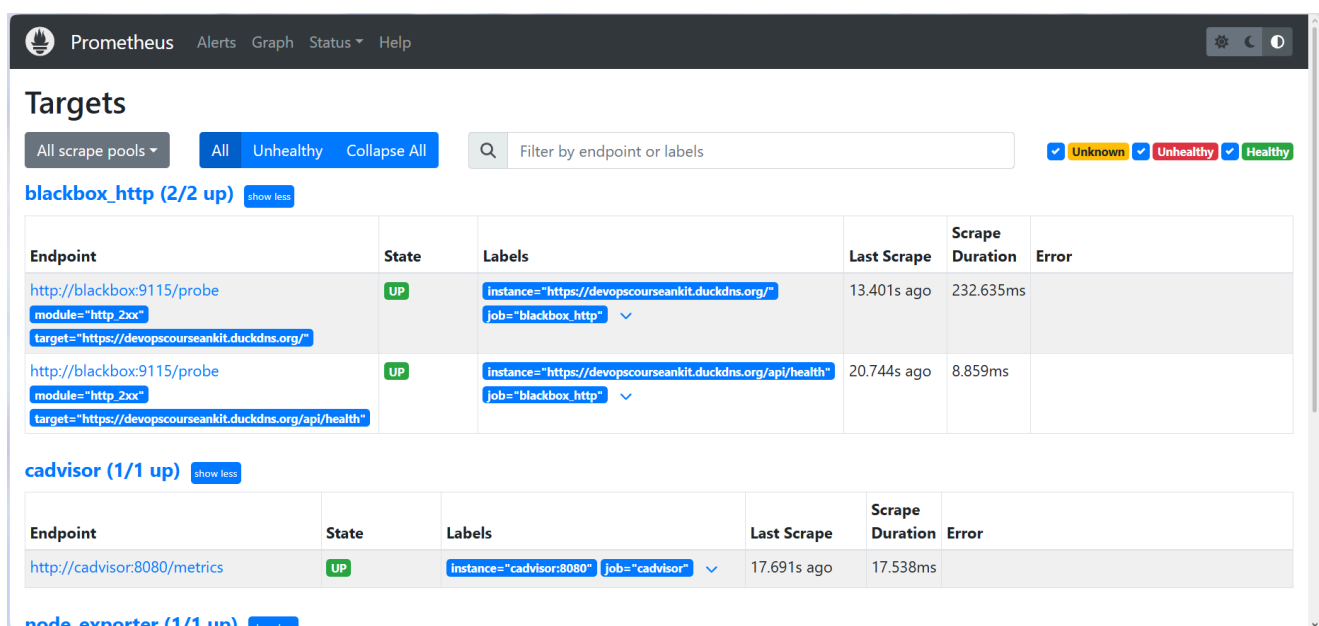
```
# Deploy the monitoring stack
ansible-playbook observability.yml
```

```
# Access:
```

```
# Prometheus: http://<YOUR_SERVER_IP>:9090/targets
```

```
# Grafana:      http://<YOUR_SERVER_IP>:3000/
```

Prometheus Targets Page — All Up



The screenshot shows the Prometheus web interface. The 'Targets' page is active, displaying a list of scrape jobs. The interface includes a navigation bar with 'Prometheus', 'Alerts', 'Graph', 'Status', and 'Help'. Below the navigation bar, there are tabs for 'All', 'Unhealthy', and 'Collapse All', along with a search bar and status filters (Unknown, Unhealthy, Healthy). The main content area shows three sections: 'blackbox_http (2/2 up)', 'cadvisor (1/1 up)', and 'node_exporter (1/1 up)'. Each section contains a table with columns for Endpoint, State, Labels, Last Scrape, Scrape Duration, and Error.

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://blackbox:9115/probe module="http_2xx" target="https://devopscourseankit.duckdns.org/"	UP	instance="https://devopscourseankit.duckdns.org/" job="blackbox_http"	13.401s ago	232.635ms	
http://blackbox:9115/probe module="http_2xx" target="https://devopscourseankit.duckdns.org/api/health"	UP	instance="https://devopscourseankit.duckdns.org/api/health" job="blackbox_http"	20.744s ago	8.859ms	
http://cadvisor:8080/metrics	UP	instance="cadvisor:8080" job="cadvisor"	17.691s ago	17.538ms	

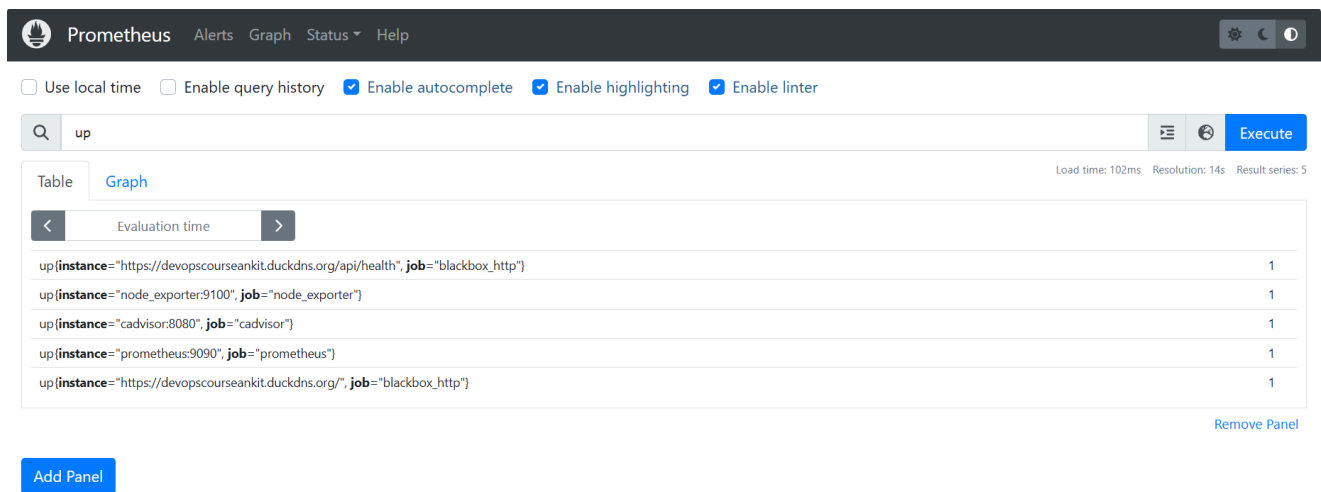
The Targets page (/targets) shows all configured scrape jobs. From the screenshot:

- **blackbox_http (2/2 up)** — both external probes are working

- `https://devopscourseankit.duckdns.org/` — scraping via Blackbox, last scrape 13.4s ago, scrape duration 232ms
- `https://devopscourseankit.duckdns.org/api/health` — scraping via Blackbox, scrape duration 8.8ms
- `cadvisor (1/1 up)` — container metrics being collected, scrape duration 17.5ms
- `node_exporter (1/1 up)` — host metrics being collected

All states show `UP` (green). This is the primary verification step after deploying the observability stack.

Prometheus Query — `up` Metric



The screenshot shows the Prometheus web interface. At the top, there's a navigation bar with 'Prometheus', 'Alerts', 'Graph', 'Status', and 'Help'. Below this, there are checkboxes for 'Use local time', 'Enable query history', 'Enable autocomplete', 'Enable highlighting', and 'Enable linter'. The main query input field contains 'up'. To the right of the input field are icons for 'Execute' and 'Refresh'. Below the input field, there are tabs for 'Table' and 'Graph'. The 'Table' tab is selected, and it shows a table with 5 rows of data. Each row represents a scrape target and its status (1 for up, 0 for down). The table has columns for the query, the status, and the evaluation time. The status for all targets is 1.

Query	Status
<code>up{instance="https://devopscourseankit.duckdns.org/api/health", job="blackbox_http"}</code>	1
<code>up{instance="node_exporter:9100", job="node_exporter"}</code>	1
<code>up{instance="cadvisor:8080", job="cadvisor"}</code>	1
<code>up{instance="prometheus:9090", job="prometheus"}</code>	1
<code>up{instance="https://devopscourseankit.duckdns.org/", job="blackbox_http"}</code>	1

Running the query `up` in the Prometheus expression browser shows the health of every scrape target. A value of `1` = target is up and scraping successfully.

From the screenshot, all 5 targets return `1` :

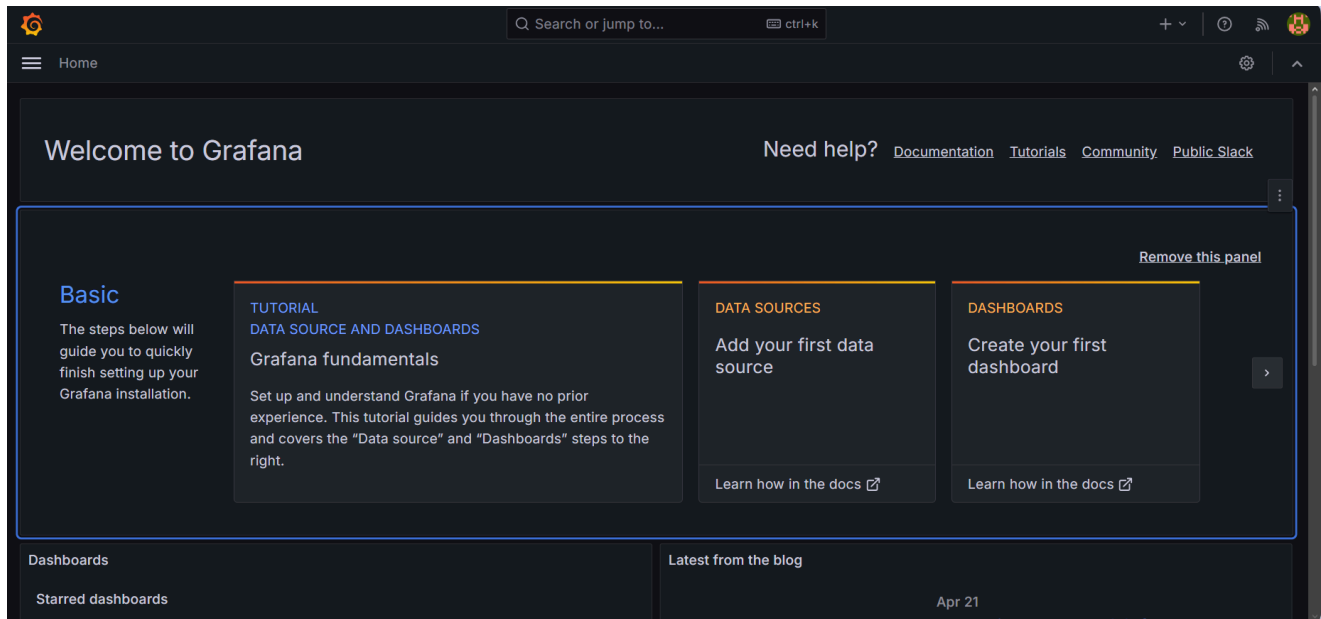
- `up{instance="https://devopscourseankit.duckdns.org/api/health", job="blackbox_http"} = 1`
- `up{instance="node_exporter:9100", job="node_exporter"} = 1`
- `up{instance="cadvisor:8080", job="cadvisor"} = 1`
- `up{instance="prometheus:9090", job="prometheus"} = 1`
- `up{instance="https://devopscourseankit.duckdns.org/", job="blackbox_http"} = 1`

Other useful verification queries:

```
probe_success          # 1 = external probe succeeded, 0 = failed
probe_duration_seconds # How long the HTTP probe took
probe_http_status_code # What status code the probe received
```

Grafana Setup and Dashboards

Grafana Welcome Screen



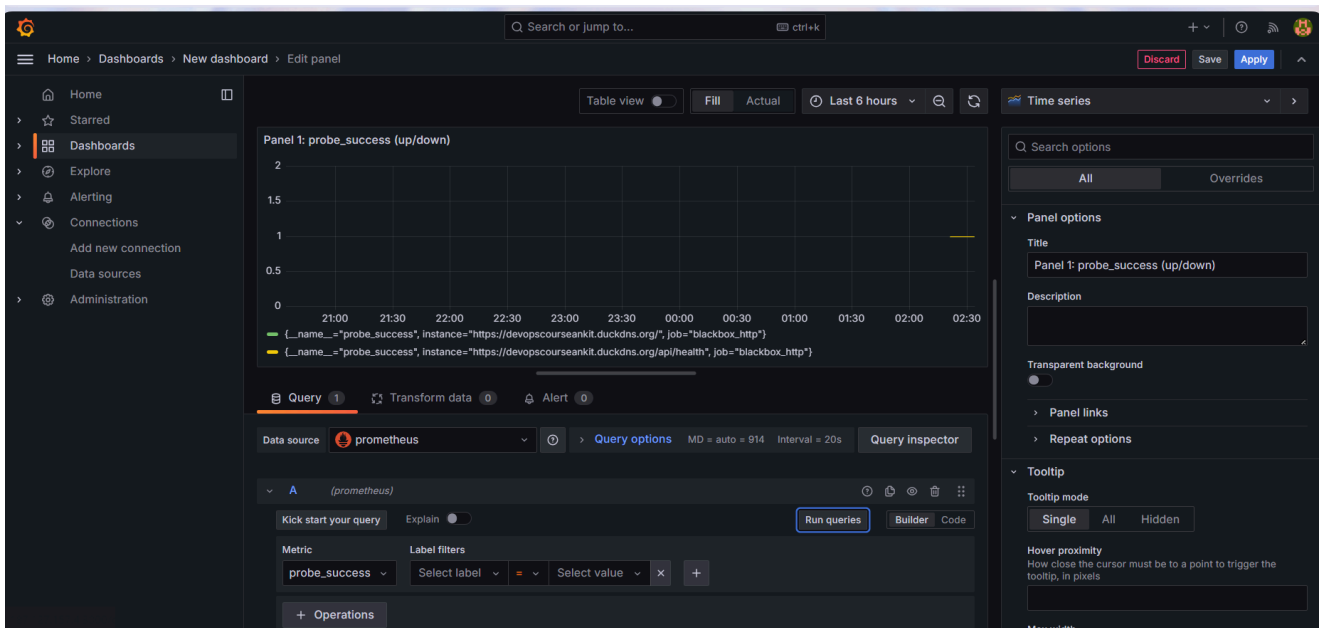
When you first access Grafana at `http://<YOUR_SERVER_IP>:3000/`, you see the welcome screen with three steps:

1. **Tutorial** — Grafana fundamentals walkthrough
2. **Data Sources** — add your first data source (Prometheus)
3. **Dashboards** — create your first dashboard

Step 1: Add Prometheus as a Data Source

- Go to **Connections** → **Data Sources** → **Add** → **Prometheus**
- URL: `http://prometheus:9090` (container name works because both are on the monitoring network)
- Click "Save & Test" — should show "Data source is working"

Step 2: Create a Dashboard with `probe_success`

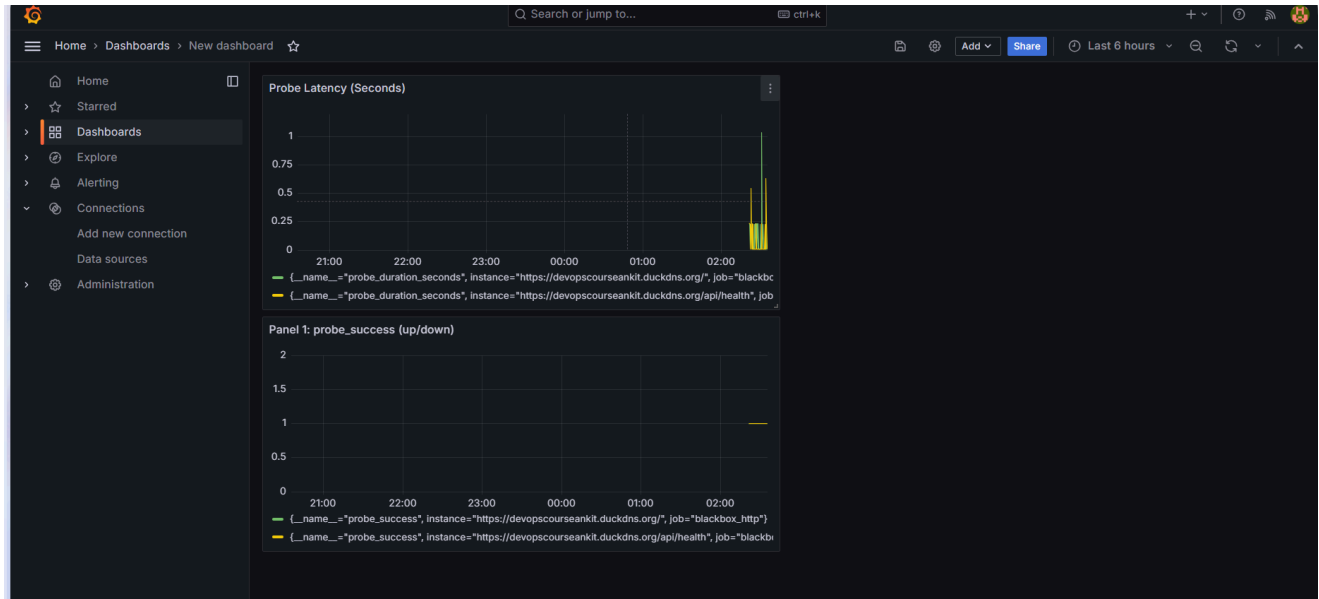


From the screenshot:

- **New Dashboard → Add Panel**
- Panel title: "Panel 1: probe_success (up/down)"
- Data source: Prometheus
- Metric: `probe_success`
- The graph shows two time series (green and yellow lines):
 - `probe_success` for `https://devopscourseankit.duckdns.org/` (green)
 - `probe_success` for `https://devopscourseankit.duckdns.org/api/health` (yellow)
- Both lines are at value 1 — both endpoints are up and healthy
- The panel shows "Last 6 hours" of data

Key architectural note: Grafana is getting the data from Prometheus only. It is purely a visualization layer — it does not store anything itself.

Step 3: Add a Second Panel for Probe Latency



The final dashboard with two panels:

- **Top panel — Probe Latency (Seconds):** using `probe_duration_seconds` — shows how long each external probe took in seconds. The spike visible at ~02:00 corresponds to when the k6 load test ran, causing higher latency.
- **Bottom panel — `probe_success` (up/down):** the same up/down visualization as before

Both panels show the same two instances (green = main URL, yellow = API health endpoint).

The Complete Observability Data Flow

```
Prometheus → Scrape/Metrics
    ↓
Node_Exporter      (Host CPU / RAM / Disk)
    ↓
cadvisor           (Container-level details)
    ↓
blackbox exporter  → probes public HTTPS URLs
```

Full end-to-end:

1. Node Exporter exposes host metrics at `:9100/metrics`
cAdvisor exposes container metrics at `:8080/metrics`
Blackbox Exporter probes `devopscourseankit.duckdns.org` every 15s

2. Prometheus scrapes all targets every 15 seconds → stores as time-series
3. Grafana connects to Prometheus (prometheus:9090 as data source) → runs PromQL queries → renders dashboards visually
4. Engineers see: CPU trends, container stats, endpoint availability in one place

Grafana k6 — Load Testing

What is Load Testing?

Load testing is the process of understanding how your system handles multiple users hitting it simultaneously. An application might work perfectly for 1 user but fail for 1,000. Load testing reveals this before real users discover it.

Three things load testing validates:

1. **Latency** — Does response time increase as traffic increases? Measured via P95 and P99 (the time within which 99% of requests complete)
2. **System Stability** — Does the app crash or return errors under stress?
3. **Resource Behavior** — How does CPU/RAM respond to load? (Visible in Grafana dashboards while k6 runs)

What is k6?

k6 is a scriptable load generator. Unlike basic tools, it lets you write test logic in **JavaScript** to simulate real-world traffic patterns — different endpoints, realistic wait times between requests, gradual traffic ramp-up.

Key metrics k6 collects:

- `http_req_duration p(50)` — median response time (half of requests faster than this)
- `http_req_duration p(95)` — 95th percentile (95% of requests faster than this)
- `http_req_duration p(99)` — 99th percentile (worst-case experience for most users)

- `http_req_failed` — error rate

The Test File (`ops/tests/k6/statusboard.js`)

```
import http from 'k6/http';
import { check, sleep } from 'k6';

// Traffic ramp-up configuration
export const options = {
  stages: [
    { duration: '15s', target: 5 }, // Gradually increase to
    5 virtual users over 15s
    { duration: '30s', target: 15 }, // Ramp up to 15 virtual
    users over 30s
    { duration: '15s', target: 0 }, // Ramp down to 0 (cool-
    down period)
  ],
};

const BASE_URL = __ENV.BASE_URL ||
  'https://devopscourseankit.duckdns.org';

export default function () {
  // Test the health endpoint
  const healthRes = http.get(`${BASE_URL}/api/health`);
  check(healthRes, {
    'health check status is 200': (r) => r.status === 200,
  });

  // Test the main page
  const webRes = http.get(`${BASE_URL}/`);
  check(webRes, {
    'web page status is 200': (r) => r.status === 200,
  });

  sleep(1); // 1 second pause between iterations (simulates
  real user behavior)
}
```

Why the gradual ramp-up? Hitting a server with 100 users simultaneously from zero is unrealistic and causes an artificial spike. Gradual ramp-up reveals at exactly what traffic level performance starts to degrade — which is much more actionable information.

Deploying the k6 Test File via Ansible

Add to `deploy.yml` :

```
- name: Ensure k6 directory exists on VPS
  file:
    path: /opt/statusboard/ops/tests/k6
    state: directory

- name: Sync k6 tests to VPS
  copy:
    src: ops/tests/k6/
    dest: /opt/statusboard/ops/tests/k6/
```

Running the Load Test from the VPS

```
# SSH into the VPS
ssh root@<YOUR_SERVER_IP>
cd /opt/statusboard

# Run k6 using Docker (no installation needed on the server)
docker run --rm -i \
  -e BASE_URL="https://devopscourseankit.duckdns.org" \
  -v "$PWD:/work" \
  -w /work \
  grafana/k6 run ops/tests/k6/statusboard.js
```

Reading the k6 Output

```
      /\      Grafana  /--/
     /\  /  \   | \  _  /  /
    /  \ /    \ |  | /  /  --\
   /    \      |  (  |  (  )  |
  /  -----  \ |  | \  \  \----/
```

```
execution: local
script: ops/tests/k6/statusboard.js
```

```
scenarios: 1 scenario, 15 max VUs, 1m30s max duration
* default: Up to 15 looping VUs for 1m0s over 3 stages
```

```
✓ health check status is 200
✓ web page status is 200

checks.....: 100.00% ✓ 458 ✗ 0
http_req_duration.....: avg=207ms min=8ms med=12ms
                        max=1.2s p(90)=890ms p(95)=1.1s
http_reqs.....: 229    3.8/s
vus.....: 1      min=1 max=15
```

Interpreting results:

- `checks 100.00%` — all requests returned expected status codes, no errors
- `p(95)=1.1s` — 95% of requests completed within 1.1 seconds. If your performance target is 500ms, this tells you the system struggles under 15 concurrent users
- `http_reqs 3.8/s` — the system handled 3.8 requests per second

Watching Grafana while k6 runs: Open your Grafana CPU usage dashboard simultaneously. You will see the CPU spike visibly as virtual users ramp up — this is exactly the resource behavior k6 is designed to expose.